# Deep Code Analysis 2026-01-11 Algorithms And Patterns

**Summary:** Dataclass-based state (from AI-DnD) Property-based computed values (hp_percent, modifiers) Adapter pattern (4-stat to 6-stat conversion) Save system with checksums (integrity...

**Generated:** 2026-01-12 05:14 UTC | **Ideas Extracted:** 52

## What Happened

> Dataclass-based state (from AI-DnD) Property-based computed values (hp_percent, modifiers) Adapter pattern (4-stat to 6-stat conversion) Save system with checksums (integrity checking) Quest system with objectives (progress tracking).

ctavolazzi/AI-DnD (HIGHEST PRIORITY) Clone repository Study gamemanager.py architecture Analyze pixelgame_manager.py for UI patterns Extract quest system implementation.

> Adapter Pattern: 4-stat to 6-stat conversion ✅ State Management: Centralized with dataclasses ✅ Save System: JSON with checksums ✅ Quest System: Objective progress tracking ✅ Combat System: Turn-based with initiative ✅.

Deep analysis of actual source code from D&D 5e repositories reveals critical algorithms, data structures, and patterns that can directly benefit WAFT's D&D mechanics implementation. This document focuses on actionable code patterns rather than high-level descriptions.

WAFT Integration Opportunity: CRITICAL: This solves the 4-stat vs 6-stat problem Class-based stat derivation (WIS/CHA from class) Modifier calculation: (stat - 10) // 2 AC calculation: 10 + dex_modifier Proficiency bonus: 2 + (level // 4).

Algorithm: Attack Roll Calculation Formula: d20 + attackbonus >= targetAC Attack Bonus: proficiency-bonus + STRmodifier (melee) or DEX_modifier (ranged) Critical Hit: Natural 20 on d20.

> Key Concepts (from web search): CommonTemplate: Shared fields (abilities, currency) Attributes-Fields: AC, Initiative, Movement, HP TraitsFields: Size, Damage Immunities/Resistances, Conditions.

result = d20.roll("1d20+5") print(result.total) # Total: 15 print(result.crit) # Critical hit detection.

> WAFT Integration: Use d20 library for all dice rolling Supports complex expressions Built-in critical hit detection Tree-based representation for parsing.

def makeattackroll(attack_modifier: int, advantage: bool = False, disadvantage: bool = False) -> tuple: """Make an attack roll. Returns (total, hit, critical)."""" if advantage and not disadvantage: roll = max(d20.roll("1d20").total, d20.roll("1d20").total) elif disadvantage and not advantage: roll = min(d20.roll("1d20").total, d20.roll("1d20").total) else: roll = d20.roll("1d20").total.

Critical Algorithms for WAFT: Ability Modifier: (score - 10) // 2 Proficiency Bonus: Level-based table lookup AC Calculation: Base 10 + DEX, modified by armor HP Calculation: Hit die + CON modifier per level Attack Roll: d20 + proficiency + ability modifier Saving Throw: d20 + ability + proficiency (if proficient) vs DC.

@staticmethod def attackroll(advantage: bool = False, disadvantage: bool = False) -> tuple: """Make attack roll. Returns (total, iscritical)."""" if advantage: roll = max(d20.roll("1d20").total, d20.roll("1d20").total) elif disadvantage: roll = min(d20.roll("1d20").total, d20.roll("1d20").total) else: roll = d20.roll("1d20").total.

> d20 - Dice rolling engine Installation: pip install d20 Used by: Avrae, many D&D tools Features: Complex expressions, critical detection.

def makeattackroll(self, advantage: bool = False) -> tuple: """Make attack roll. Returns (total, hit, critical)."""" roll, iscritical = self.rolld20(advantage) strmod = self.abilitymodifier(self.strength) prof = self.proficiencybonus() total = roll + strmod + prof return (total, total, iscritical).

> def rolld20(self, advantage: bool = False) -> tuple: """Roll d20. Returns (result, is_critical)."""" if advantage: roll = max(d20.roll("1d20").total, d20.roll("1d20").total) else: roll = d20.roll("1d20").total return (roll, roll == 20).

Key Pattern: Centralized state management with dataclasses.

Key Pattern: Stackable items with quantity tracking.

Key Pattern: Adapter pattern for D&D stat conversion.

> Key Pattern: JSON serialization with integrity checking.

total = roll + attack_modifier critical = (roll == 20) return (total, total, critical).

> iscritical = (roll == 20) return (roll, iscritical).

Generated: 2026-01-11 Analysis Type: Deep Code & Algorithm Exploration Purpose: Extract reusable algorithms, patterns, and code structures for WAFT integration.

WAFT Integration Opportunity: Use similar dataclass pattern for WAFT's Being state Property-based computed values (hppercent, manapercent) Status effects as list (extensible) Ability scores as dictionary (flexible).

savedata = { "version": self.SAVEVERSION, "slot": slot, "name": savename, "createdat": now, "updatedat": now, "metadata": metadata or {}, "gamedata": game_data }.

> WAFT Integration Opportunity: Save file integrity checking Version tracking for migration Metadata separation from game data Backup system before overwrite.

def update(self, count: int = 1) -> bool: """Update progress. Returns True if objective completed.""" self.currentcount = min(self.currentcount + count, self.requiredcount) if self.currentcount >= self.required_count: self.completed = True return self.completed.

## Additional Details

WAFT Integration Opportunity: hitdie: Used for HP calculation (d12 for Barbarian) proficiencychoices: Player choices during character creation savingthrows: Which saves the class is proficient in startingequipment: Default equipment for new characters.

WAFT Integration Opportunity: Template-based data model Mixin pattern for shared fields Derived data calculation (prepareDerivedData()).

Algorithm: Dice Expression Parsing Pattern: NdM+K where N=dice count, M=sides, K=modifier Example: 5d20+6 = roll 5d20, add 6 Advanced: 2d20kh1 = keep highest of 2d20.

> WAFT Integration: Combine both approaches Use dataclass for state Use dictionary for ability scores (flexible) Status effects as list (extensible).

WAFT Integration: Slot-based equipment system Stackable items with quantity Gold as separate currency Capacity limits.

> High Priority: StatsAdapter from AI-DnD - Solves 4-stat to 6-stat conversion SaveSystem from AI-DnD - Game state persistence with checksums QuestTracker from AI-DnD - Objective-based quest system d20 library - Dice rolling engine (already exists).

Medium Priority: InventoryState from AI-DnD - Stackable items, equipment slots CharacterState from AI-DnD - HP, mana, status effects GameStateSerializer from AI-DnD - State serialization patterns.

WAFT Integration: Use 5e-database as reference data source Parse JSON structures for game mechanics Build WAFT's D&D system on top of this data.

@staticmethod def ability_modifier(score: int) -> int: """Calculate ability modifier.""" return (score - 10) // 2.

> @staticmethod def proficiency_bonus(level: int) -> int: """Calculate proficiency bonus.""" return 2 + ((level - 1) // 4).

@staticmethod def armorclass(dexmod: int, armortype: str = "none", armorbase: int = 0) -> int: """Calculate AC.""" if armortype == "none": return 10 + dexmod elif armortype == "light": return armorbase + dexmod elif armortype == "medium": return armorbase + min(dexmod, 2) elif armortype == "heavy": return armorbase return 10.

> @staticmethod def spellsavedc(spellcastingmod: int, proficiency: int) -> int: """Calculate spell save DC.""" return 8 + spellcastingmod + proficiency.

@staticmethod def roll(expression: str) -> int: """Roll dice expression.""" result = d20.roll(expression) return result.total.

Create D&D 5e module in WAFT: src/waft/core/dnd5e/ directory stats.py - Stat calculations dice.py - Dice rolling wrapper character.py - Character state combat.py - Combat mechanics.

Integrate 5e-database data: Download or reference JSON files Create data loaders Build character creation from class data.

> WAFT Integration: Study dice rolling patterns Character sheet data structures Command-based interface patterns.

WAFT Integration: Command parsing patterns Game state modification via commands Combat encounter setup.

> 5e-bits/5e-database Clone repository Study JSON structure Create data loaders Map data to WAFT models.

Create waft.core.dnd5e module Implement stat calculation functions Integrate d20 library Build character creation system Implement combat mechanics.

Pattern: Command-based game mechanics via hashtags.

Quest System: Objective-based tracking Progress calculation Reward distribution.

> Combat System: Turn-based initiative Attack roll vs AC Damage application Status effects.

5e-SRD - System Reference Document Legal: Open Gaming License Content: Core rules and mechanics.

> foundryvtt/dnd5e Study actor data model Analyze template system Understand derived data calculation.

Status: Deep code analysis complete. Ready for implementation phase.

> Next Action: Begin implementing D&D 5e module in WAFT using extracted algorithms and patterns.

### Content Breakdown

| Type | Count |
|---|---|
| Decisions | 0 |
| Insights | 13 |
| Actions | 9 |
| Concepts | 29 |
| Questions | 1 |